Sistema de Gestión de Personal para el Club de Fútbol New Team.

- **Descripción del proyecto:**

El club de fútbol New Team, reconocido por contar con jugadores y entrenadores de alto rendimiento, ha identificado la necesidad de modernizar su sistema de gestión de personal. Para ello, se ha propuesto desarrollar una aplicación que permita manejar de manera eficiente la información de sus empleados, tanto entrenadores como jugadores, facilitando operaciones básicas como la creación, lectura, actualización y eliminación de registros (CRUD), así como la realización de consultas específicas sobre el personal.

El sistema está diseñado para ser altamente configurable y versátil, soportando la carga y exportación de datos en múltiples formatos (CSV, XML, JSON y binario), lo que permite una fácil integración con otras herramientas y sistemas. Además, se ha implementado un servicio de caché LRU (Least Recently Used) con un límite de 5 elementos para optimizar el acceso a los datos más frecuentemente utilizados.

· Objetivos del proyecto:

- **Gestión eficiente del personal:** Permitir la administración de la información de jugadores y entrenadores, incluyendo datos básicos como nombre, apellidos, fecha de nacimiento, salario, país de origen, y detalles específicos como la posición en el campo (para jugadores) o la especialización (para entrenadores).

- **Validación de datos**: Asegurar que los datos ingresados en el sistema cumplan con los requisitos establecidos, evitando errores y conflictos en el almacenamiento.

- **Soporte multiplataforma:** Garantizar que el sistema pueda ser ejecutado en diferentes entornos, gracias a su empaquetado en un archivo JAR configurable mediante ficheros de configuración.

- **Consultas avanzadas**: Ofrecer un conjunto de consultas predefinidas que permitan analizar y extraer información relevante del personal, como el jugador más joven, el entrenador con mayor salario, o la media de goles anotados por los delanteros.

- **Documentación y pruebas**: Entregar un proyecto completamente documentado, con pruebas unitarias y de integración que aseguren su correcto funcionamiento.

- Alcance del proyecto:

El sistema está dirigido específicamente al personal del club de fútbol New Team, incluyendo jugadores y entrenadores. No se contempla, en esta fase, la gestión de otros tipos de empleados (por ejemplo, personal administrativo o médico). Además, el sistema está diseñado para ser utilizado a través de una interfaz de consola, con opciones para cargar y guardar datos en diferentes formatos.

- Beneficios esperados:

  - **Optimización de procesos**: Reducción del tiempo necesario para gestionar la información del personal.
  - **Acceso rápido a datos**: Gracias a la implementación de la caché LRU, se mejora la eficiencia en el acceso a los datos más utilizados.
  - **Flexibilidad**: Soporte para múltiples formatos de entrada y salida, lo que facilita la integración con otros sistemas.
  - **Toma de decisiones informada**: Las consultas avanzadas permiten al club analizar el rendimiento y las características de su personal de manera detallada.

- **Presentación del equipo y reparto de tareas:**

El equipo consta de tres integrantes:

- Eduardo Miranda Garcia.
- Javier Izquierdo Portillo.
- David Rayón Gómez.

- **Análisis de riesgos**

Riesgos identificados:

1) Retrasos en la entrega:

   - **Descripción**: Debido a la complejidad del sistema, especialmente en la implementación de la caché LRU y el soporte para múltiples formatos de datos

(CSV, XML, JSON, binario), existe el riesgo de que el equipo no cumpla con los plazos establecidos.

- **Mitigación**:
    - Uso de metodologías ágiles (como Scrum) para dividir el trabajo en sprints y realizar entregas incrementales.
    - Reuniones diarias (daily stand-ups) para monitorear el progreso y ajustar el plan de trabajo según sea necesario.

2) Problemas de integración entre módulos:

- **Descripción**: Dado que el sistema está compuesto por varios módulos (gestión de datos, caché LRU, manejo de formatos, etc.), existe el riesgo de que estos no funcionen correctamente al integrarse.
- **Mitigación**:
    - Pruebas de integración continuas para asegurar que los módulos funcionen correctamente juntos.
    - Uso de herramientas de integración continua (CI/CD) como Jenkins o GitHub Actions.
    -

3) Errores en la validación de datos:

- **Descripción**: Si la validación de datos no se implementa correctamente, podrían ingresarse datos incorrectos o inconsistentes en el sistema, lo que afectaría la calidad de la información.
- **Mitigación**:
    - Implementación de pruebas unitarias exhaustivas para los módulos de validación de datos.
    - Revisión manual de los casos de prueba para asegurar que se cubran todos los escenarios posibles.

4) Bajo rendimiento del sistema:

- **Descripción**: El uso de la caché LRU podría no ser suficiente para garantizar un rendimiento óptimo si el volumen de datos es muy grande o si la implementación no es eficiente.

- **Mitigación**:
    - Pruebas de carga para evaluar el rendimiento del sistema bajo diferentes condiciones.
    - Optimización del código y revisión de la implementación de la caché LRU.

5) Falta de documentación o documentación incompleta:
    - **Descripción**: Si la documentación no es clara o está incompleta, podría dificultar el mantenimiento futuro del sistema o la comprensión del mismo por parte de nuevos desarrolladores.
    - **Mitigación**:
        - Asignar un miembro del equipo específicamente para la documentación.
        - Revisión periódica de la documentación para asegurar que esté actualizada y sea clara.

6) Problemas de compatibilidad con formatos de datos:
    - **Descripción**: El soporte para múltiples formatos de datos (CSV, XML, JSON, binario) podría generar problemas de compatibilidad, especialmente si los formatos no están bien definidos o si hay inconsistencias en los datos.
    - **Mitigación**:
        - Pruebas exhaustivas para cada formato de datos, incluyendo casos extremos (por ejemplo, archivos muy grandes o con datos faltantes).
        - Uso de bibliotecas robustas y bien documentadas para el manejo de formatos (por ejemplo, Jackson para JSON, OpenCSV para CSV).

7) Falta de cobertura de pruebas:
    - **Descripción**: Si las pruebas no cubren una parte significativa del código, podrían pasar errores inadvertidos a la versión final del sistema.
    - **Mitigación**:
        - Uso de herramientas de cobertura de código (como JaCoCo) para medir la cobertura de las pruebas.
        - Establecer un objetivo mínimo de cobertura de código (por ejemplo, 85%).

8) Problemas de comunicación en el equipo:

- **Descripción**: Si no hay una comunicación efectiva entre los miembros del equipo, podrían surgir malentendidos o duplicación de esfuerzos.

· **Mitigación**:

  - Uso de herramientas de comunicación como Slack o Microsoft Teams para facilitar la colaboración.
  - Reuniones regulares para alinear expectativas y resolver dudas.

- **Especificación de requisitos funcionales, no funcionales y de información**

En este apartado se detallan los requisitos que el sistema de gestión de personal para el club de fútbol New Team debe cumplir. Estos requisitos se dividen en tres categorías: funcionales, no funcionales y de información.

· **Requisitos funcionales**

Los requisitos funcionales describen las funcionalidades que el sistema debe ofrecer. A continuación, se enumeran los principales:

1) <u>Gestión de personal:</u>

El sistema debe permitir la creación, lectura, actualización y eliminación (CRUD) de registros de empleados (jugadores y entrenadores).

Cada empleado debe tener un identificador único generado automáticamente por el sistema.

2) <u>Carga y exportación de datos:</u>

El sistema debe soportar la carga de datos desde ficheros en formatos CSV, XML, JSON y binario.

El sistema debe permitir la exportación de datos a los mismos formatos (CSV, XML, JSON y binario).

3) <u>Validación de datos:</u>

El sistema debe validar los datos ingresados para evitar errores, como fechas inválidas, campos vacíos o formatos incorrectos.

4) <u>Consultas específicas:</u>

El sistema debe permitir realizar consultas predefinidas sobre los datos.

5) Caché LRU:

El sistema debe implementar un servicio de caché LRU con un límite máximo de 5 elementos para optimizar el acceso a los datos más utilizados.

6) Interfaz de consola:

El sistema debe ofrecer un menú de consola con las siguientes opciones:

- Cargar datos desde fichero.
- Crear miembro del equipo.
- Actualizar miembro del equipo.
- Eliminar miembro del equipo.
- Copiar datos a fichero.
- Realizar consultas específicas.

· **Requisitos no funcionales.**

Los requisitos no funcionales describen cómo el sistema debe comportarse y las restricciones que debe cumplir:

1) Rendimiento:

El sistema debe ser capaz de manejar un mínimo de 1,000 registros de empleados sin degradación significativa del rendimiento.

Las consultas deben ejecutarse en un tiempo máximo de 2 segundos.

2) Configurabilidad:

El sistema debe ser configurable mediante ficheros de configuración, permitiendo ajustar parámetros como el límite de la caché LRU y el formato de salida por consola.

3) Localización:

Toda salida de información por consola debe estar localizada según lo indicado en el fichero de configuración (por ejemplo, soporte para múltiples idiomas).

4) Portabilidad:

El sistema debe ser ejecutable en cualquier entorno que soporte Java, empaquetado en un archivo JAR.

5) Pruebas:

El código debe estar completamente testeado, con una cobertura mínima del 85% en pruebas unitarias y de integración.

6) Documentación:

El sistema debe estar completamente documentado, incluyendo comentarios en el código y un manual de usuario.

· **Requisitos de información**

Los requisitos de información describen los datos que el sistema debe manejar y cómo deben ser estructurados:

- Datos de empleados.
- Datos específicos de entrenadores.
- Área de especialización (entrenador de porteros, entrenador principal, entrenador asistente).
- Datos específicos de jugadores.

- **Diagrama de arquitectura de software**

El sistema de gestión de personal para el club de fútbol New Team sigue una arquitectura basada en capas, que separa las responsabilidades del sistema en módulos independientes para facilitar el mantenimiento y la escalabilidad. A continuación, se describe la arquitectura y se incluye un diagrama conceptual.

· **Descripción de la arquitectura:**

- Capa de presentación (Frontend):
- Interfaz de consola: Es la capa con la que interactúa el usuario. Permite cargar datos, realizar operaciones CRUD y ejecutar consultas específicas.
- Menú de opciones: Ofrece un menú interactivo para que el usuario seleccione las operaciones que desea realizar.

· **Capa de lógica de negocio (Backend):**

- Gestión de empleados: Contiene la lógica para crear, leer, actualizar y eliminar registros de empleados (jugadores y entrenadores).
- Caché LRU: Implementa un servicio de caché para optimizar el acceso a los datos más utilizados.
- Validación de datos: Se encarga de validar que los datos ingresados cumplan con los requisitos establecidos.
- Consultas específicas: Ejecuta las consultas predefinidas sobre los datos, como el jugador más joven o el entrenador con mayor salario.

· **Capa de persistencia de datos:**

- Almacenamiento de datos: Gestiona la carga y exportación de datos en formatos CSV, XML, JSON y binario.
- Ficheros de configuración: Permite configurar parámetros del sistema, como el límite de la caché LRU y el formato de salida por consola.

· **Capa de pruebas:**

- Pruebas unitarias: Verifican el correcto funcionamiento de cada módulo de forma independiente.
- Pruebas de integración: Aseguran que los módulos funcionen correctamente al integrarse.

- **Diagrama de arquitectura:**

```
+----------------------+
|  Interfaz de         |
|    Consola           | <--- Capa de presentación
+---------------------+
          |
          v
+---------------------+
| Gestión de           |
| Empleados            | <--- Capa de lógica de negocio
| Caché LRU            |
| Validación           |
| Consultas            |
+---------------------+
          |
```

```
          v
+-------------------------+
| Almacenamiento          | <--- Capa de persistencia de datos
| de Datos                |
| (CSV, XML, JSON, |
|  Binario)               |
+-------------------------+
          |
          v
+-------------------------+
| Pruebas Unitarias | <--- Capa de pruebas
| y de Integración  |
+-------------------------+
```

- **Herramientas tecnológicas usadas**

Para el desarrollo del sistema de gestión de personal, se han utilizado las siguientes herramientas y tecnologías:

· **Lenguaje de programación:**

  - Kotlin: Se eligió Kotlin por su interoperabilidad con Java, su sintaxis más concisa y moderna, y su capacidad para reducir errores comunes en el código. Además, Kotlin es ideal para el desarrollo de aplicaciones de consola y ofrece un alto rendimiento.

· **Entorno de desarrollo:**

  - IntelliJ IDEA: Se utilizó IntelliJ IDEA como entorno de desarrollo integrado (IDE) debido a su excelente soporte para Kotlin, su capacidad para gestionar proyectos complejos y su integración con herramientas como Git, Maven y pruebas unitarias.

· **Herramientas de desarrollo:**

  - Git y GitHub: Para el control de versiones y la colaboración en equipo. Se siguió el modelo de trabajo con GitFlow para gestionar las ramas y las integraciones.
  - Trello: Para la gestión de tareas y la organización del trabajo en equipo.
  - Maven: Para la gestión de dependencias y la construcción del proyecto.

· **Formatos de datos:**

- CSV: Para la carga y exportación de datos en formato de texto plano.
- XML: Para la carga y exportación de datos en formato estructurado.
- JSON: Para la carga y exportación de datos en formato ligero y fácil de leer.
- Binario: Para la carga y exportación de datos en formato binario, optimizado para el almacenamiento y la velocidad.

· **Herramientas de pruebas:**

- JUnit: Para la implementación de pruebas unitarias y de integración.

· **Otras herramientas:**

- Docker: Para empaquetar la aplicación y garantizar su ejecución en diferentes entornos.

- **Justificación de elecciones: Kotlin, IntelliJ IDEA y GitHub**

1. Kotlin (Lenguaje de programación)

Razones de la elección:

1. Interoperabilidad con Java:
   - Kotlin es completamente interoperable con Java, lo que permite reutilizar bibliotecas y frameworks existentes sin problemas. Esto es especialmente útil en proyectos donde se necesita integrar con sistemas o herramientas basadas en Java.
2. Sintaxis concisa y moderna:
   - Kotlin reduce la verbosidad del código en comparación con Java, lo que facilita su lectura y mantenimiento. Características como las extensiones de funciones, lambdas y tipos de datos avanzados permiten escribir código más limpio y eficiente.
3. Seguridad en el código:
   - Kotlin incluye características como tipos no nulos (non-null types), que ayudan a prevenir errores comunes en tiempo de ejecución, como los NullPointerException. Esto mejora la robustez del código y reduce la necesidad de validaciones manuales.
4. Adopción creciente en la industria:
   - Kotlin es el lenguaje preferido para el desarrollo de aplicaciones Android y está ganando popularidad en otros ámbitos, como el desarrollo backend y de consola.

Su comunidad está en crecimiento, lo que garantiza un buen soporte y una amplia gama de recursos.

2. IntelliJ IDEA (Entorno de desarrollo)

Razones de la elección:

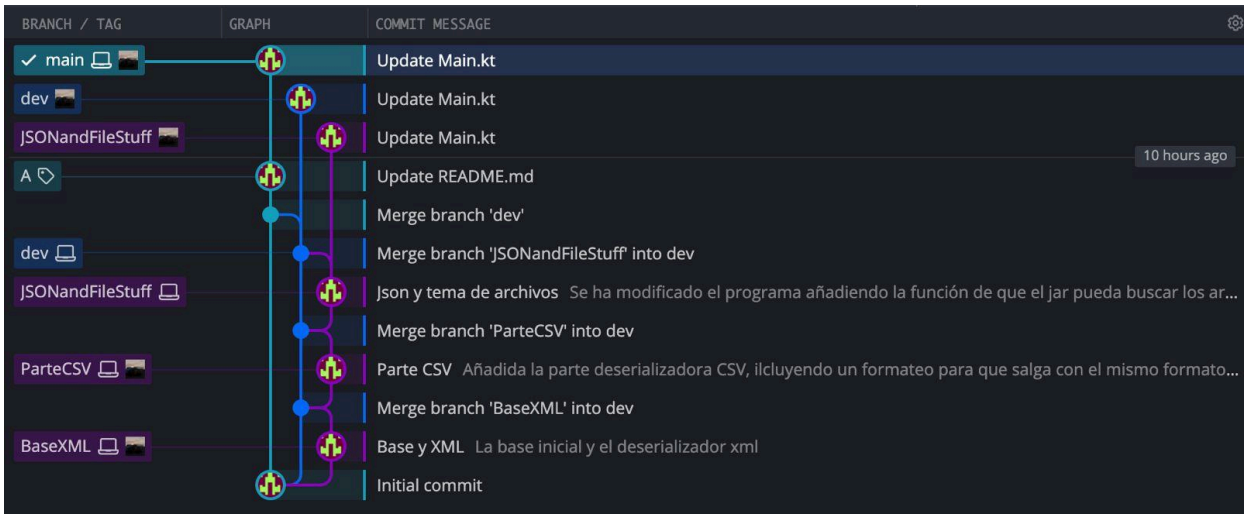
1. Soporte nativo para Kotlin:
    - IntelliJ IDEA es el IDE oficial para Kotlin, lo que garantiza un soporte óptimo para el lenguaje, incluyendo autocompletado inteligente, refactorización avanzada y detección de errores en tiempo real.
2. Herramientas integradas:
    - Incluye herramientas para pruebas, depuración, control de versiones y gestión de dependencias, lo que aumenta la productividad del equipo. Ofrece integración con Maven y Gradle para la gestión de dependencias y la construcción del proyecto.
3. Productividad:
    - Características como el autocompletado inteligente, la navegación rápida entre archivos y la refactorización avanzada permiten a los desarrolladores trabajar de manera más eficiente. Es especialmente útil para proyectos complejos con múltiples módulos.
4. Comunidad y soporte:
    - IntelliJ IDEA tiene una gran comunidad de usuarios y una amplia documentación, lo que facilita la resolución de problemas y el aprendizaje.

3. GitHub (Control de versiones y colaboración)

Razones de la elección:

1. Popularidad y adopción:
    - GitHub es la plataforma más utilizada en la industria para el control de versiones y la colaboración en proyectos de software. Su amplia adopción garantiza una gran cantidad de recursos, tutoriales y soporte.
2. Integración con herramientas de desarrollo:
    - GitHub se integra perfectamente con herramientas como IntelliJ IDEA, Maven y JUnit, lo que facilita la gestión del código, las pruebas y la integración continua.
3. Colaboración en equipo:
    - GitHub permite trabajar en equipo de manera eficiente mediante el uso de ramas, pull requests y revisiones de código. Esto facilita la colaboración y el seguimiento de los cambios en el proyecto.
4. GitHub Actions:
    - GitHub ofrece integración con GitHub Actions, lo que permite automatizar tareas como la ejecución de pruebas, la construcción del proyecto y el despliegue, mejorando la eficiencia del flujo de trabajo.

- **Captura del grafico de donde se muestre el modelo de flujo de trabajo Git Flow:**



Este gráfico muestra el flujo de trabajo GitFlow utilizado en el proyecto. Se inicia con un commit inicial y se crean ramas como dev para desarrollo. Las ramas específicas (JSONandFileStuff,ParteCSV, BaseXML) se fusionan en dev para integrar funcionalidades como manejo de JSON, CSV y XML. Finalmente, dev se fusiona en main para consolidar los cambios. Las actualizaciones en Main.kt y README.md reflejan mejoras continuas.

Principios SOLID aplicados y ejemplos de su implementación y uso en el código

1. Principio de Responsabilidad Única: Una clase debe tener una única responsabilidad, para evitar que sea demasiado compleja. La clase JSONReadWrite tiene varias funciones (mainReadJSON, existeJSON y legibleJSON, analizadorJSON, formateadorJSON) que cumplen este principio, pues cada una de ellas realizan una única misión.

   Existen otras clases para los ficheros CSV y XML que cumplen también con este principio SOLID, al tener unas funciones semejantes que en el caso de JSON.

```java
package prog.trbe2.jsonstuff

import kotlinx.serialization.json.Json
import java.io.File

class JSONReadWrite(private val archivoJSON: File) {
    fun mainReadJSON(archivoJson: File) {
        existeJSON(archivoJSON)
        legibleJSON(archivoJSON)
    }
    fun existeJSON(archivoJSON: File): Boolean {
        var resultFind: Boolean = false
        print("🔄 Buscando el archivo JSON ")
        when {
            archivoJSON.exists() -> {
                println("✅ Archivo encontrado!")
                resultFind = true
            }
            !archivoJSON.exists() -> {
                throw IllegalArgumentException("⚠️ Archivo no encontrado!")
                resultFind = false
            }
        }
        return resultFind
    }
    fun legibleJSON(archivoJSON: File): Boolean {
        var resultFind: Boolean = false
        print("🔄 Comprobando el archivo JSON ")
        when {
            archivoJSON.canRead() -> {
                println("✅ Archivo listo para usar!")
                resultFind = true
            }
            !archivoJSON.canRead() -> {
                throw IllegalArgumentException("⚠️ Archivo no legible, bórralo y añade uno que sí lo sea")
                resultFind = false
            }
        }
```

```java
    fun analizadorJSON(archivoJSON: File) {
        val json = Json { ignoreUnknownKeys = true }
        val personalList: List<Personal> = json.decodeFromString(archivoJSON.readText())
        personalList.forEach { personal ->
            println(formateadorJSON(personal))
        }
    }

    fun formateadorJSON(personal: Personal): String {
        return "Personal(id=${personal.id}, tipo=${personal.rol}, nombre=${personal.nombre}, apellidos=${personal.apellidos}, fechaNacimie
    }
}
```

2. Principio Abierto/Cerrado: dicta que las diferentes entidades, como son las clases y las funciones, deben permitir añadir nuevas funcionalidades sin alterar el código existente.

   Este principio no se cumple en nuestro proyecto, al no haber empleado clases abstractas ni interfaces.

3. Principio de Sustitución de Liskov: Los objetos de una clase hija que hereda de una clase padre debe poder implementar las funciones de esta última sin ningún problema.

No se cumple debido a que no empleamos la herencia ni el polimorfismo.

4. Principio de Segregación de Interfaces: se recomienda el uso de varias interfaces sencillas que una grande y compleja.

   Las diferentes clases tienen métodos específicos para tareas relacionadas con la lectura de los ficheros en diferentes formatos, por lo que cumplen con este principio.

5. Principio de Inversión de Dependencias: En lugar de que los módulos de alto nivel (los que definen la lógica principal del programa) dependan directamente de los módulos de bajo nivel (los que implementan detalles concretos), ambos deben depender de abstracciones.

   En este proyecto no tenemos en cuenta este principio, pues no recurrimos a la abstracción para que haya una relación indirecta entre los módulos de alto y bajo nivel.

- **Explicación teórico-práctica de cada prueba o conjunto de test relevantes realizados:**

1. Se Identifican los componentes a probar:
   - Gestión de empleados (jugadores y entrenadores).
   - Validación de datos (nombres, fechas, salarios).
   - Consultas específicas (jugador más joven, entrenador con mayor salario…).
   - Manejo de formatos (CSV, XML, JSON, binario).
   - Caché LRU.

2. Estructura de pruebas:
   - Se crean clases de prueba en src/test/kotlin (por ejemplo, JugadorTest, EntrenadorTest).
   - Usamos @Test para métodos de prueba y aserciones (assertEquals, assertTrue, etc.).

3. Tipos de pruebas:
   - Validación: Verificamos que los datos cumplan con los requisitos.
   - Consultas: Probamos que las consultas devuelven resultados correctos.
   - Formatos: Probamos carga y exportación de datos en CSV, XML, JSON y binario.
   - Caché: Verificamos que la caché LRU funcione correctamente.

4. Ejecución y cobertura:
   - Ejecutamos pruebas con IntelliJ IDEA, Maven o Gradle.

- **Estimación económica:**

1. Costo de recursos humanos:
- Horas de trabajo: Cada estudiante dedica aproximadamente 150 horas al proyecto.
- Tarifa por hora: Tarifa simbólica de 10€/hora (como referencia).

3 estudiantes * 50 horas * 10€/hora = 1,500€

2. Costo de herramientas:
- IntelliJ IDEA: Versión gratuita (por ser estudiante).
- GitHub: Gratuito para estudiantes.
- Otras herramientas: Docker, Maven, JUnit, etc., son gratuitas.

Total herramientas: 0€ (gratuitas para estudiantes).

3. Costo de infraestructura:
- Servidores: No se necesitan servidores externos; el proyecto se ejecuta localmente.
- Dominio y hosting: No hay.

Total infraestructura: 0€.

4. Gastos indirectos:
- Electricidad y espacio: Trabajamos desde casa o clase, sin costos adicionales.
- Comunicación: Herramientas gratuitas como Discord o WhatsApp.

Total gastos indirectos: 0€.

5. Costo total del proyecto:
Costo recursos humanos: 1,500€
Costo herramientas: 0€
Costo infraestructura: 0€
Gastos indirectos: 0€
Total estimado: 1,500€